Comparative Analysis of Machine Translation Models

RNN, Encoder–Decoder, Additive Attention, and Multiplicative Attention for English–Spanish Translation

Rijan Ghimire
Roll No: ACE080BCT055

Department of Computer Engineering
Advanced College of Engineering and Management
Tribhuvan University, Institute of Engineering (IOE)

Dataset: ManyThings.org Anki Repository — English-Spanish
((`spa-eng.zip`, 144,215 sentences pairs) — `random_state = 55`)

August 21, 2026

Abstract

Neural Machine Translation (NMT) is a method of automatically translating text feom one languages to another using deep learning. Different NMT architectures however deal with information and long-range dependencies in various ways. This study offers a comparison of four neural machine translation architectures for Spanish translation: a Vanilla Recurrent Neural Network (RNN), a basic Encoder-Decoder architecture, an Encoder-Decoder with Additive Attention and an Encoder-Decoder with Multiplicative Attention.

The experiment uses the Spanish bilingual dataset from the ManyThings.org Anki Language Dataset repository. After processing and filtering a reproducible sample of the dataset is chosen using `random_state = 55`. The same data preparation process, dataset split, training setup and evaluation method are used for all four models to ensure a comparison.

The models are tested using measures like validation loss, BLEU score, perplexity and training performance. Also a qualitative analysis of the translations is done to look at differences in accuracy word order keeping the meaning and how well the models handle longer or more complex sentences. The study pays attention to the issues with fixed-context architectures and looks into how Additive and Multiplicative Attention mechanisms help the decoder find relevant information, from the source sequence during translation.

The results of this study are meant to give an understanding of the pros and cons of different NMT architectures and how attention mechanisms contribute to better machine translation performance.

1 Introduction

Machine Translation (MT) is the automatic process of translating text from one language to another while preserving its meaning and producing a fluent sentence in the target language. It plays an important role in communication and information exchange across different languages [1].

Earlier approaches, such as Rule-Based Machine Translation (RBMT) and Statistical Machine Translation (SMT), relied on manually designed rules, dictionaries, or statistical relationships from bilingual data. Although useful, these methods often required significant manual effort and had difficulty handling complex linguistic structures.

Recent advances in deep learning have led to Neural Machine Translation (NMT), which learns the relationship between source and target languages directly from parallel data. NMT treats translation as an end-to-end sequence-to-sequence learning problem [2, 3].

This project compares four NMT architectures: a Simple RNN, a basic Encoder–Decoder model, an Encoder–Decoder with Additive Attention, and an Encoder–Decoder with Multiplicative Attention. All models are trained and evaluated under the same experimental conditions to ensure a fair comparison.

Neural Machine Translation (NMT)

A typical NMT system uses an Encoder–Decoder architecture, where the encoder processes the source sentence and the decoder generates the translation one token at a time [3].

A key limitation of the basic Encoder–Decoder model is its fixed-size context vector, which may lose important information when handling longer or more complex sentences [**?**]. Attention mechanisms address this issue by allowing the decoder to focus on different parts of the source sentence during translation [4].

This study compares Additive Attention [4] and Multiplicative Attention [5], which differ in how they calculate alignment between the encoder and decoder. The models are evaluated using BLEU scores, loss, and qualitative analysis to examine how architectural differences affect translation quality.

2 Methodology

This study follows an experimental approach to compare four Neural Machine Translation (NMT) architectures for English–Spanish translation. The models are trained and evaluated under comparable experimental conditions so that differences in performance can primarily be attributed to architectural design. The overall methodology consists of dataset preparation, text preprocessing, vocabulary construction, model implementation, training, and evaluation.

2.1 Dataset

The English–Spanish bilingual dataset used in this study was obtained from the ManyThings.org Anki Language Dataset repository. The `spa-eng.zip` file contains 144,215 English–Spanish sentence pairs from the Tatoeba project, with each sample consisting of an English sentence and its corresponding Spanish translation.

The dataset was selected because it provides a large parallel corpus while remaining suitable for training and comparing the four NMT architectures within the available computational resources.

Table 1: Summary of the English–Spanish Dataset

Property	**Value**
Dataset Source	ManyThings.org Anki Language Dataset
Language Pair	English → Spanish
Dataset File	`spa-eng.zip`
Total Sentence Pairs	144,215
Random State	55

2.2 Data Preprocessing

The raw bilingual dataset was preprocessed before training. The procedure included removing unnecessary metadata, handling missing or duplicate entries, converting text to lowercase, normalizing Unicode characters, and tokenizing sentences into words and punctuation.

The following special tokens were used:

<pad>, <sos>, <eos>, <unk>

The <sos> and <eos> tokens mark the beginning and end of a sentence, while <pad> is used to make sequences within a batch the same length. The <unk> token represents words not included in the vocabulary.

Separate English and Spanish vocabularies were created using only the training data to avoid information leakage. Words occurring below the selected minimum frequency threshold were mapped to <unk>.

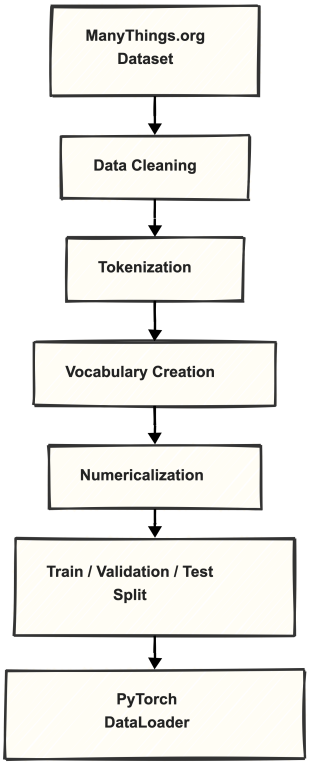

Figure 1: Overview of the data preprocessing pipeline.

2.3 Dataset Partitioning

The processed dataset was divided into training, validation, and test sets. The training set was used to optimize model parameters, the validation set was used to monitor performance during training and assist in model selection, and the test set was reserved for final evaluation.

To ensure reproducibility, dataset partitioning was performed using:

```
random_state = 55
```

The same dataset split was used for all four architectures to ensure a fair comparison.

Table 2: Experimental Dataset Partition

Dataset	Sentence Pairs	Percentage
Training	24,000	80%
Validation	3,000	10%
Testing	3,000	10%
Total	30,000	100%

2.4 Model Architectures

Four neural machine translation architectures were implemented and compared.

2.4.1 Recurrent Neural Network (RNN)

The first model uses a basic Recurrent Neural Network (RNN) as the baseline. An RNN processes the input sequence one token at a time while maintaining a hidden state that carries information from previous tokens. The hidden state is computed as:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

where x_t is the input, h_t is the hidden state, and W_x, W_h, and b are learnable parameters.

However, standard RNNs can struggle to retain information over long sequences due to vanishing gradients and limited memory.

2.4.2 Encoder–Decoder Architecture

The second model uses a sequence-to-sequence Encoder–Decoder architecture. The encoder processes the English source sequence and produces a context representation, which the decoder uses to generate the Spanish translation one token at a time.

The conditional probability of the target sequence can be expressed as:

$$P(y|x) = \prod_{t=1}^{T} P(y_t \mid y_{<t}, c)$$

where c is the context representation produced by the encoder. However, compressing the entire source sentence into a fixed-size vector can cause important information to be lost, especially for longer sentences.

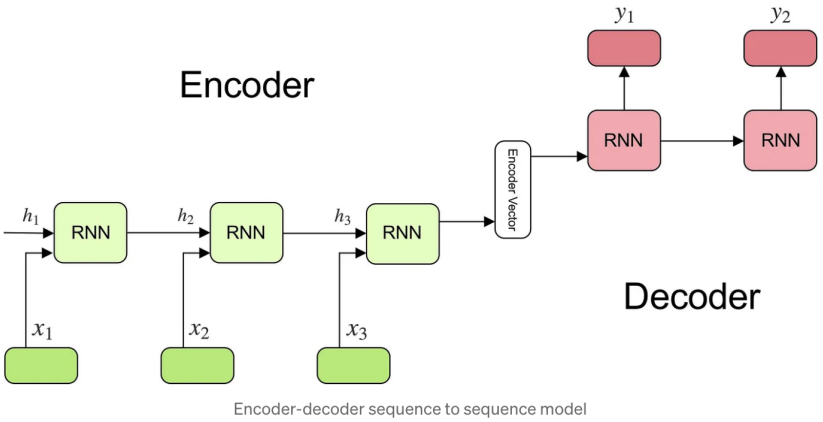

Figure 2: *Architecture of a basic sequence-to-sequence (Seq2Seq) Encoder–Decoder model using Recurrent Neural Networks (RNNs). The encoder compresses the input sequence (x_1, x_2, x_3) into a single fixed-size vector, which initializes the decoder to predict the target sequence (y_1, y_2).*

2.4.3 Encoder–Decoder with Additive Attention

The third model extends the Encoder–Decoder architecture with Additive Attention, also known as Bahdanau Attention. Instead of using a single fixed context vector, the decoder assigns attention weights to the encoder hidden states at each decoding step.

The alignment score is calculated as:

$$e_{t,i} = v_a^T \tanh(W_h h_i + W_s s_{t-1})$$

where h_i is the encoder hidden state, s_{t-1} is the previous decoder state, and W_h, W_s, and v_a are learnable parameters.

The attention weights are obtained using softmax:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}$$

The resulting context vector is:

$$c_t = \sum_i \alpha_{t,i} h_i$$

This allows the decoder to focus on the most relevant parts of the source sentence during translation.

2.4.4 Encoder–Decoder with Multiplicative Attention

The fourth model uses Multiplicative Attention, commonly known as Luong Attention. It calculates the compatibility between the decoder state and encoder hidden states using a multiplicative function.

The alignment score is given by:

$$e_{t,i} = s_t^T W_a h_i$$

where s_t is the decoder hidden state, h_i is the encoder hidden state, and W_a is a learnable weight matrix.

The attention weights are obtained using softmax:

$$\alpha_{t,i} = \text{softmax}(e_{t,i})$$

The context vector is then calculated as:

$$c_t = \sum_i \alpha_{t,i} h_i$$

This context is combined with the decoder state to generate the target token, allowing the model to focus on relevant source information during translation.

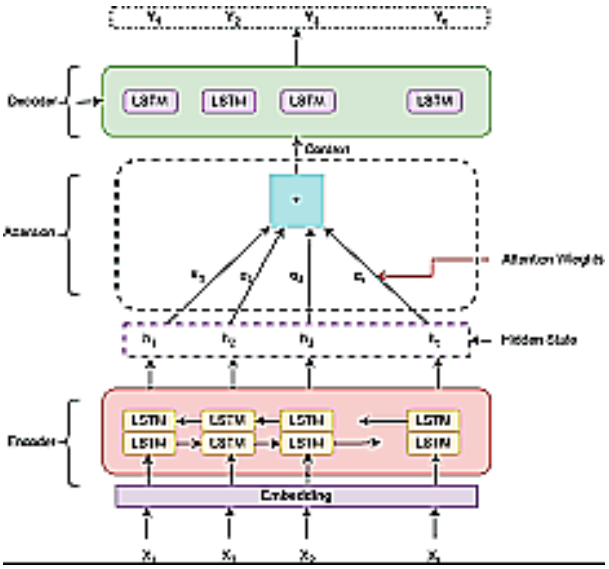

Figure 3: *Layered workflow of an Attention-based Encoder-Decoder sequence-to-sequence model. The input sequence ($x_1 \dots x_4$) passes through an embedding layer into a bidirectional LSTM encoder, yielding hidden states ($h_1 \dots h_4$). The central Attention Layer dynamically scales these states using computed alignment weights ($\alpha_1 \dots \alpha_4$) to form a pooled Context vector, which drives the decoder LSTM units to generate target sequence predictions ($Y_1 \dots Y_4$).*

3 Experimental Setup

All four Neural Machine Translation models were trained and evaluated under comparable experimental conditions. The same experimental dataset, preprocessing procedure, vocabulary construction method, train–validation–test split, and evaluation metrics were used for all models. This ensured that differences in performance could primarily be attributed to differences in model architecture.

3.1 Computing Environment

The experiments were implemented using Python and the PyTorch deep learning framework. Training was performed on a local machine using the available hardware acceleration through the Apple Metal Performance Shaders (MPS) backend when available.

3.2 Dataset Split

A reproducible random sample of 30,000 sentence pairs was selected from the filtered English–Spanish dataset using `random_state = 55`. The experimental dataset was divided into training, validation, and test sets using an 80%, 10%, and 10% split, respectively. The same split was used for all four models.

3.3 Training Configuration

The models were trained using the Adam optimizer and Cross-Entropy loss. Padding tokens were ignored during loss calculation. Teacher forcing was used during training to improve convergence, while autoregressive decoding was used during validation and testing.

Table 3: Common Training Configuration

Parameter	**Value**
Random State	55
Batch Size	64
Embedding Dimension	128
Hidden Dimension	256
Optimizer	Adam
Learning Rate	0.001
Teacher Forcing Ratio	0.5
Number of Epochs	10
Loss Function	Cross-Entropy Loss

3.4 Evaluation Metrics

The models were evaluated using validation loss, test loss, BLEU score, perplexity, training time, and qualitative analysis of generated translations. Lower loss and perplexity indicate better predictive performance, whereas a higher BLEU score indicates greater similarity between generated translations and reference translations.

4 Results

This section presents the performance of the four Neural Machine Translation architectures on the English–Spanish test dataset. The models were evaluated using test loss, perplexity, BLEU-1, and BLEU-4 scores.

4.1 Training and Validation Performance

The training and validation loss curves provide an overview of how the models learned during training. In general, the attention-based models achieved lower validation losses than the non-attention-based architectures. This indicates that allowing the decoder to dynamically access information from the encoder improved the models ability to predict the target sequence.

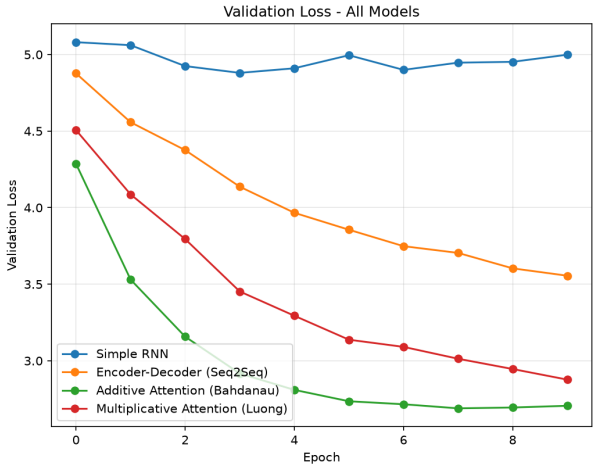

Figure 4: Training and validation loss comparison of the four NMT models.

Figure 4 illustrates the training and validation loss trajectories across 10 epochs. The **Simple RNN** shows minimal learning, remaining stalled near a loss of 5.0. The standard **Encoder–Decoder** model demonstrates steady convergence, reducing loss to 3.6. In contrast, both attention mechanisms significantly accelerate optimization: **Additive Attention** achieves the lowest overall validation loss (2.7), closely followed by **Multiplicative Attention** (2.9). This demonstrates that dynamic alignment allows the decoder to learn target mappings faster and reach lower loss bounds.

4.2 Quantitative Comparison

Table 4 compares the performance of the four models on the test dataset.

Table 4: Performance Comparison of NMT Models

Model	**Parameters**	**Test Loss**	**Test PPL**	**BLEU-1**	**BLEU-4**
Simple RNN	30,144	4.941	139.910	23.930	0.051
Encoder–Decoder	46,600	3.628	37.633	40.280	6.762
Additive Attention	62,815	2.794	16.343	57.790	21.703
Multiplicative Attention	38,042	2.970	19.496	55.190	18.760

The results show a clear improvement from the Simple RNN to the attention-based models. The Simple RNN performed the worst, with the highest loss and perplexity and a BLEU-4 score close to zero.

The Encoder–Decoder model performed noticeably better, reducing the test loss to 3.628 and increasing the BLEU-4 score to 6.762. However, both attention-based models achieved substantially stronger results.

Among all models, Additive Attention performed best, achieving the lowest test loss and perplexity and the highest BLEU-1 and BLEU-4 scores. Multiplicative Attention achieved the second-best translation performance, with a BLEU-4 score of 18.760, while using fewer parameters than the Additive Attention model. Overall, the results show that incorporating attention significantly improves translation quality.

4.3 Qualitative Translation Analysis

Table 5 shows selected translations generated by the four models. These examples help illustrate the differences in translation quality beyond the quantitative metrics.

Table 5: Examples of Generated English–Spanish Translations

English Input	Simple RNN	Encoder–Decoder	Additive Attention	Multiplicative Attention
I am a student.	tom no que que de que <unk>.	soy un.	soy estudiante estudiante.	soy estudiante.
She is reading a book in the library.	tom no que que de que <unk>.	ella es un un en un.	ella está leyendo una libro en la biblioteca.	ella leyendo una libro en la biblioteca.
We need to protect our environment.	tom no que que de que <unk>.	tenemos que <unk> <unk>.	necesitamos que nuestro deber <unk>.	necesitamos <unk> nuestro <unk>.
Can you help me solve this problem?	¿ qué qué ?	¿ me puedo hablar este este ?	¿ puede ayudarme a este problema problema ?	¿ puedes ayudarme a este problema ?

The Simple RNN produces repetitive and incomplete translations, while the Encoder–Decoder model generates more meaningful sentences but still makes structural and vocabulary errors. Both attention-based models produce translations that are generally closer to the intended meaning. In several examples, Multiplicative Attention generates the most natural output, although Additive Attention achieves better overall scores. These examples support the quantitative results and highlight the advantage of attention mechanisms in machine translation.

5 Discussion

The results show a clear pattern across the four models. The Simple RNN performed the weakest overall. Its very low BLEU-4 score and high perplexity suggest that, although it could occasionally predict individual words correctly, it struggled to maintain context and generate meaningful sentences.

The Encoder–Decoder model showed a noticeable improvement over the Simple RNN. Separating the encoding and decoding processes helped the model produce more meaningful translations. However, the model still relied on a fixed context representation, which limited its ability to retain all relevant information from longer or more complex sentences.

The biggest improvement was observed after introducing attention. Both Additive and Multiplicative Attention achieved much better results than the previous models. Additive Attention performed best overall, achieving the highest BLEU-4 score of 21.703 and the lowest test loss and perplexity. This suggests that allowing the decoder to focus on different parts of the input sentence during translation helped the model use contextual information more effectively.

Multiplicative Attention also performed well and achieved a BLEU-4 score of 18.760. Although its overall score was slightly lower than that of Additive Attention, it used fewer trainable parameters and produced some of the most natural translations in the qualitative examples. This shows that model performance should not be judged only by a single metric.

Overall, the results show that attention plays an important role in improving translation quality. It helps the model overcome some of the limitations of fixed-context architectures and produce translations that are generally more meaningful and structured.

6 Conclusion

This study compared four Neural Machine Translation architectures for English–Spanish translation: Simple RNN, Encoder–Decoder, Encoder–Decoder with Additive Attention, and Encoder–Decoder with Multiplicative Attention.

The comparison showed a clear improvement from the Simple RNN to the attention-based models. The Encoder–Decoder model improved translation quality, but the attention mechanisms provided a much larger improvement. Among the four models, Additive Attention achieved the best overall performance, while Multiplicative Attention produced competitive results with fewer trainable parameters.

The study shows that allowing the decoder to focus on relevant parts of the source sentence can significantly improve machine translation performance. Although the models in this study were trained under limited experimental settings, the results clearly demonstrate the advantage of attention-based architectures over basic recurrent and fixed-context models.

Future work could explore larger datasets, different hyperparameter settings, bidirectional encoders, and more advanced approaches such as Transformer-based architectures.

References

[1] B. N. Brunda, V. Potdar, L. Santhosh, N. Indu, and N. C. Brunda, "Comparative study of machine translation techniques," *International Journal of Advanced Research (IJAR)*, vol. 11, pp. 387–402, June 2023.

[2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, pp. 3104–3112, 2014.

[3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014.

[4] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations (ICLR)*, 2015.

[5] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1412–1421, 2015.